

Centroidal MPC for Inertial Reorientation Using Bio-inspired Tailed Robot

Yichen Wang, Yue Qin

Abstract—Inertial reorientation (IR) is capability for robots to change their base orientation without or with minimal ground interaction, a feature that is often observed in nature. Tailed robots have an advantage in inertial reorientation due to the large inertia of the tail, but the tail also poses difficulties in control. We propose a centroidal dynamics-based control strategy that reduces the computational complexity compared to full-body dynamics while maintaining the ability to consider individual link inertias, including the tail. Reduced-order centroidal dynamics is formulated, and reference trajectories are generated using offline trajectory optimization. The CM-MPC controller is then deployed to track these reference trajectories, leveraging the tail's inertia for reorientation. Simulation results are presented for two reorientation tasks, a 90° and a 180° turn, demonstrating the controller's capability to utilize tail dynamics for base orientation control. However, the results reveal limitations in achievable reorientation angles and high-frequency oscillations in control torques as the robot deviates from the reference trajectory. These findings show the effectiveness of centroidal dynamics for robot agility but also reveals the need for further optimization in the trajectory generation and MPC formulation to improve robustness and performance.

I. INTRODUCTION

Inertial reorientation (IR) refers to the ability of a robot to change its orientation without or with minimal interaction with the ground or other objects. This movement is well-utilized in the animal kingdom, enhancing aerial movements and [1]. When applied to robotics, it can increase robot agility by stabilizing the robot between ground contacts in gaits with long flight phases, rejecting sudden disturbances, or increasing the speed of the robots turns. A tail-like appendage is shown to be effective in increasing the robot's ability for inertial-reorientation [2]–[4].

However, the addition of a tail appendage makes the robot system more complex by introducing more degrees of freedoms to the system, potentially at the detriment of computation time. Fu et al. [3] uses offline trajectory optimization to bypass the computational speed problem while considering all tail vertebral inertias. Chu et al. [5] defines a metric over which a quadratic program is constructed, but it is only deployed on a simple wheel-and-tail robot.

Single rigid body (SRB) is a popular reduced-order model used in model predictive control of legged locomotion [6]. However, an SRB model lumps all link inertias of the robot together, resulting in an inability to reason about individual link inertia for inertial reorientation.

Centroidal MPC can be a good alternative to full body dynamics for tailed robot control [7]. It is both able to consider individual link inertias and is a reduced order model.

Zhou et al. [8] uses centroidal momentum dynamics in an MPC framework to control a quadruped robot, showing good performance in disturbance rejection and agility improvement. However, the robot used does not have a tail, and a large tail inertia may affect the dynamics significantly.

It is the goal of this project to implement a centroidal MPC controller for inertial reorientation of a bipedal tailed robot. In this paper, we present a robot model, the formulation of the centroidal momentum dynamics, and the MPC controller. We then show simulation results of the controller in two inertial reorientation tasks, and discuss the performance and limitations of the controller.

II. METHODS

A. Robot Model

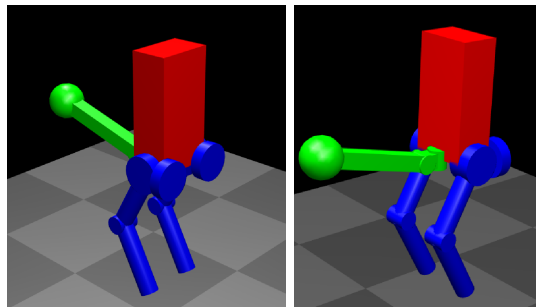


Fig. 1. Robot model with $q_j = [0 \ .25 \ 0 \ .5 \ -1 \ 0 \ .5 \ -1]^\top$. The left picture shows the robot from the front, and the right shows the robot from the back. X-axis points forward, Y-axis points right, and the Z-axis points up.

We use a bipedal tailed robot model. The robot has eight actuated degrees of freedom, denoted $q_j \in \mathbb{R}^8$: two for the tail, and three for each leg. The tail is modelled as two sequentially actuated joints for pitch and yaw, placed close to the torso, with a long bar and a heavy counterweight connected at the end. Each leg is modeled as three sequential joints for adduction-abduction, hip extension-flexion, and knee-extension-flexion. Additionally, the model includes six floating base joints, $q_b \in \mathbb{R}^6$, to simulate free movement in three dimensions: three linear joints in the x, y, and z direction, and three rotating joints in the x, y, and z axis.

We used the software package *Spatial V2* [9] to model and visualize this model, shown in figure 1. The parent link, joint properties, mass, and inertia of each link are denoted in Table I, where $p(j)$ denotes the parent link of link i , Position denotes the translation portion of the transform from the predecessor frame to the current link frame, ${}^i\mathbf{X}_p(i)$, CoM

Pos refers to the center of mass position in the current link frame, $I \in \mathbb{R}^{3 \times 3}$ refers to the diagonalized link inertia around the CoM. All link frames are constructed with identical orientation to the world frame when $q_j = \mathbf{0}^8$, so frame orientations are omitted. Link 0, or the Torso, is connected to ground via six sequential floating-base joints, using the order $[Px Py Pz Rx Ry Rz]$. q_b and q_j are constructed with the same order as above. We further define $q = [q_b^\top q_j^\top]^\top \in \mathbb{R}^{14}$ to be the concatenation of floating-base and actuated joints.

TABLE I
TABLE OF ROBOT PARAMETERS.

i	Name	$p(i)$	Axis	Frame Pos.	M	CoM Pos.	$\text{diag}(I) \times 10^4$
0	Torso	n/a	n/a	[0 0 0]	1	[0 0 .2]	[94 83 27]
1	Tail-Y	0	z	[-0.05, 0, 0]	.05	[-0.025 0 0]	[0 0.104 0.104]
2	Tail-P	1	y	[-0.05, 0, 0]	.5	[-0.27 0 0]	[0 25.5 25.5]
3	AbAd-R	0	x	[0, -.075, 0]	.3	[0 0 0]	[0.625 22.75 22.75]
4	Hip-R	3	y	[0, 0, 0]	.75	[0 0 -0.02]	[18.25 38.8 21.8]
5	Knee-R	4	y	[0, 0, -0.2]	.1	[0 0 -0.1]	[3.33 3.33 0]
6	AbAd-L	0	x	[0, .075, 0]	.3	[0 0 0]	[0.625 22.75 22.75]
7	Hip-L	6	y	[0, 0, 0]	.75	[0 0 -0.02]	[18.25 38.8 21.8]
8	Knee-L	7	y	[0, 0, -0.2]	.1	[0 0 -0.1]	[3.33 3.33 0]

For this model, we did not implement ground contact or gravitation forces, in order to more closely examine the inertial reorientation performance of the controller and robot. The robot can be seen as being suspended in a zero-g environment with no outside forces.

B. Trajectory Optimization Formulation

Reference trajectories for online CM-MPC controller are generated using offline trajectory optimization. We decided to use simple Euler integration for the dynamics, with $dt = 0.02s$, time horizon of $2s$, and $N_{hor} = 100$. We formulated the trajectory optimization problem as the following NLP

$$\arg \min_{X, U} \sum_{n=1}^{N_{hor}-1} \tau^\top \text{diag}(w_1) \tau + \varepsilon_x^\top \text{diag}(w_2) \varepsilon_x \quad (1)$$

subject to

$$\varepsilon_x = x_n - x_{des,n} \quad (2)$$

$$x_{n+1} = x_n + \dot{x}_n(x_n, u) * dt \quad (3)$$

$$x_n < \text{Lower Bound} \quad (4)$$

$$x_n > \text{Upper Bound}, \quad (5)$$

where $u_i \in \mathbb{R}^8$ is desired joint torques at the i th knot point,

$$U = [u_1 \ u_2 \ \dots \ u_{N_{hor}-1}] \quad (6)$$

$$x_n = [q_{bn}^\top \ q_{jn}^\top \ \dot{q}_{bn}^\top \ \dot{q}_{jn}^\top]^\top \quad (7)$$

$$X = [x_1 \ x_2 \ \dots \ x_{N_{hor}}]. \quad (8)$$

Dynamics in (3) are encoded using the function `FDcrb` in the package `Spatial V2` [9], which uses a composite-rigid-body algorithm, and the NLP is implemented with `CasAdi` [10] and solved using `IPOPT` [11].

The solved trajectories are used as desired state trajectories for the MPC controller in section II-D. The trajectories are

interpolated from 50Hz to 250Hz using the 1-D interpolation function `griddedInterpolant`.

C. Centroidal Matrix Formulation

The centroidal momentum, $h_G \in \mathbb{R}^6$ of a robot refers to the sum of all link momenta, projected to the center of mass frame of the robot [12]. We will first define the concatenated spatial velocity of all links as $v = [v_1 \ v_2 \ \dots \ v_8]^\top$, and the spatial velocity of a specific link as $v_i \in \mathbb{R}^6$, and find the relation

$$v_i = \begin{bmatrix} \omega_i \\ \nu_i \end{bmatrix} = {}^i \mathbf{X}_p(i) v_{p(i)} + \Phi_i \dot{q}_i \quad (9)$$

where $\omega_i \in \mathbb{R}^3$ is the angular velocity, $\nu_i \in \mathbb{R}^3$ is the linear velocity, and $\Phi_i \in \mathbb{R}^6$ is the motion subspace matrix, which selects the axis in which a joint acts on. For example, a revolute joint in the y direction has $\Phi_{Ry} = [0 \ 1 \ 0 \ 0 \ 0 \ 0]^\top$.

We can shuffle (9) to get (10), the difference of velocity between a link and its predecessor.

$$v_i - {}^i \mathbf{X}_p(i) v_{p(i)} = \Phi_i \dot{q}_i \quad (10)$$

Then, we find the relationship between v and $\dot{q}$ to be

$$Pv = \Phi \dot{q} \quad (11)$$

$$P_{ij} = \begin{cases} \mathbf{1}^{6 \times 6} & , j = i \\ -{}^i \mathbf{X}_p(i) v_{p(i)} & , j = p(i) \\ \mathbf{0}^{6 \times 6} & , \text{else} \end{cases} \quad (12)$$

$$\Phi = \begin{bmatrix} \Phi_1 & \mathbf{0}^6 & \dots & \mathbf{0}^6 \\ \mathbf{0}^6 & \Phi_2 & \dots & \mathbf{0}^6 \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}^6 & \mathbf{0}^6 & \dots & \Phi_8 \end{bmatrix} \quad (13)$$

where $P \in \mathbb{R}^{84 \times 84}$ is a nonstandard incidence matrix, and $\Phi \in \mathbb{R}^{84 \times 14}$ is the diagonalized joint selection matrix.

We can also find the relation $h = Iv$ between $h \in \mathbb{R}^{84}$, the system momentum matrix and $I \in \mathbb{R}^{84 \times 84}$, with

$$h = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_8 \end{bmatrix} \quad (14)$$

$$I = \begin{bmatrix} I_1 & \mathbf{0}^6 & \dots & \mathbf{0}^6 \\ \mathbf{0}^6 & I_2 & \dots & \mathbf{0}^6 \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0}^6 & \mathbf{0}^6 & \dots & I_8 \end{bmatrix} \quad (15)$$

where $h_i \in \mathbb{R}^6$ is the link momentum in the world frame, and $I_i \in \mathbb{R}^{6 \times 6}$ is the link inertia. In order to find the centroidal momentum, we must transform the link momentum from the world frame into the CoM frame, and then sum it

$$h_G = \sum_{i=1}^8 {}^i \mathbf{X}_G^T h_i = \sum_{i=1}^8 {}^i \mathbf{X}_G^T I_i v_i \quad (16)$$

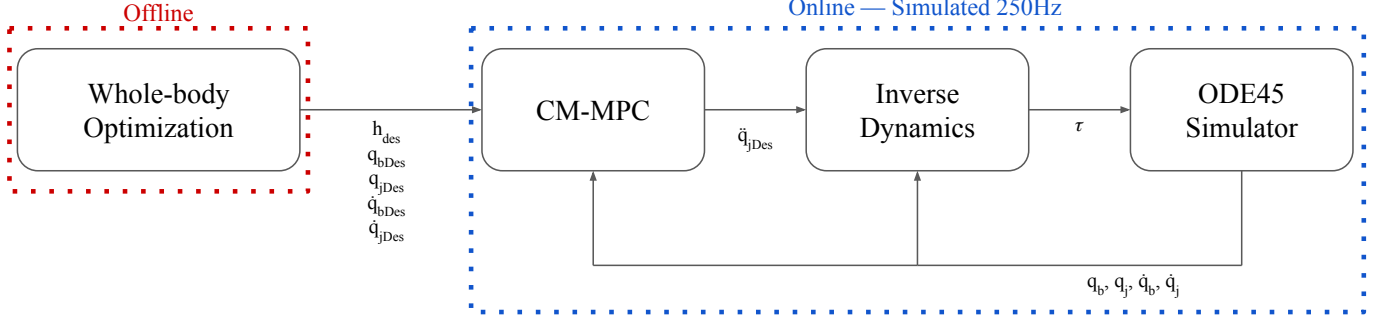


Fig. 2. Illustration of the overall system. An offline trajectory optimization feeds reference trajectories into a simulated online CM-MPC and Inverse Dynamics controller to control the robot.

Using matrix manipulation to substitute the summation, we find

$$h_G = \underbrace{\mathbf{X}_G^T \mathbf{I} \mathbf{P}^{-1} \Phi}_{A_G(q)} \dot{q} \quad (17)$$

where $\mathbf{X}$ is the projection matrix from centroidal coordinates to all link coordinates,

$$\mathbf{X}_G = \begin{bmatrix} {}^1\mathbf{X}_G^T \\ {}^2\mathbf{X}_G^T \\ \vdots \\ {}^8\mathbf{X}_G^T \end{bmatrix} \quad (18)$$

From (17), we can see that the centroidal momentum is linear in terms of joint velocity $\dot{q}$. which means we can find the centroidal momentum matrix $A_G(q) \in \mathbb{R}^{6 \times 14}$, for which $h_G = A_G(q)\dot{q}$. The fact that the h_G is linear in $\dot{q}$ is useful in the centroidal MPC formulation, as it simplifies the optimization problem significantly.

D. Centroidal MPC Formulation

The centroidal MPC uses linearized centroidal dynamics as a reduced order model for the robot in the NLP. We formulated the centroidal dynamics similarly to Zhou et al. [8], with some changes to fit the robot model and centroidal momentum derivation.

Using (17), we can split q into q_b and q_j to find

$$h_G = \underbrace{[A_b(q) \ A_j(q)]}_{A_G} \begin{bmatrix} q_b \\ q_j \end{bmatrix}, \quad (19)$$

where $A_b(q) \in \mathbb{R}^{6 \times 6}$ and $A_j(q) \in \mathbb{R}^{6 \times 8}$. We can rearrange (19) into

$$\dot{q}_b = \underbrace{\left[\begin{matrix} A_b^{-1}(q) & A_b^{-1}(q)A_j(q) \\ \widetilde{A}_b(q) & \widetilde{A}_j(q) \end{matrix} \right]}_{\widetilde{A}_G(q)} \begin{bmatrix} h \\ \dot{q}_j \end{bmatrix}. \quad (20)$$

Algorithm 1 Main To, Simulation, and Control Process

Input $X_{des}, w_{1,TO}, w_{2,TO}, w_{1,MPC}, w_{2,MPC}$

$dt_{TO} \leftarrow 0.02$
 $N_{hor,TO} \leftarrow 100$
 $dt_{MPC} \leftarrow 0.004$
 $N_{hor,MPC} \leftarrow 50$

function MAKEAUGMENTEDSTATE(x)
 $[q_b, q_j, \dot{q}_b, \dot{q}_j] \leftarrow \text{deal}(x)$
 $q \leftarrow [q_b; q_j]$
 $\dot{q} \leftarrow [\dot{q}_b; \dot{q}_j]$
 $h \leftarrow A_G(q)\dot{q}$ **return** $[h, q_b, q_j, \dot{q}_j]$
end function

$x_i \leftarrow \mathbf{0}^{28}$
 $x_{i,aug} \leftarrow \text{makeAugmentedState}(x_i)$
 $[X_{ref}] \leftarrow \text{TO}(X_{des}, w_{1,TO}, w_{2,TO})$
for $i < dt_{TO} \times N_{hor,TO} / dt_{MPC}; i \leftarrow i + 1$ **do**
 $u_a \leftarrow \text{CM-MPC}(x_{i,aug}, X_{ref}, w_{1,MPC}, w_{2,MPC})$
 $\tau \leftarrow \text{InvDynamics}(u_a, x_i)$
 $[t_{sim}; x_{sim}] = \text{ODE45}(x_i, \tau)$
 $x_i \leftarrow x_{sim}(:, 0)$
 $x_{i,aug} \leftarrow \text{makeAugmentedState}(x_i)$
end for

We can define an augmented state $x_a = [h^T \ q_b^T \ q_j^T \ \dot{q}_j^T]^T$. Using (20) and x_a , we can formulate continuous time augmented state centroidal dynamics as:

$$\begin{bmatrix} \dot{h} \\ \dot{q}_b \\ \dot{q}_j \\ \ddot{q}_j \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \widetilde{A}_b(q) & \mathbf{0} & \mathbf{0} & \widetilde{A}_j(q) \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{I} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}}_{H_G(q)} \begin{bmatrix} h \\ q_b \\ q_j \\ \dot{q}_j \end{bmatrix} + \underbrace{\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{I} \end{bmatrix}}_{G_G} u_a, \quad (21)$$

where $u_a = \ddot{q}_{j,des}$ is the desired joint accelerations. Working under the assumption that q closely follows q_{des} , as calculated in the trajectory optimization, we can substitute the reference

trajectories into (21):

$$\dot{x}_a = H_G(q_{des})x_a + G_G u_a. \quad (22)$$

We then discretize (22):

$$x_{i+1} = \widetilde{H}_{G_i} x_i + \widetilde{G}_{G_u} u_i \quad (23)$$

$$\widetilde{H}_{G_i} = H_G(q_{des,i}) * dt + \mathbf{I}^{28 \times 28} \quad (24)$$

$$\widetilde{G}_G = G_G \times dt. \quad (25)$$

We can then formulate an MPC using (23) as the dynamics constraint:

$$\arg \min_{X_a, U_a} \sum_{n=1}^{N_{hor}-1} u_n^\top \text{diag}(w_1) u_n + \varepsilon_n^\top \text{diag}(w_2) \varepsilon_n \quad (26)$$

subject to

$$\varepsilon_n = x_n - x_{des,n} \quad (27)$$

$$x_{i+1} = \widetilde{H}_{G_i} x_i + \widetilde{G}_{G_u} u_i \quad (28)$$

$$x_n < \text{Lower Bound} \quad (29)$$

$$x_n > \text{Upper Bound}. \quad (30)$$

Although (26) is linear, we still opted to solve it using Ipopt and CasAdi, similar to the trajectory optimization problem mentioned above. For each solution of the NLP, we use only u_i for the inverse dynamics controller. X_a and U_a are used to warm start the next MPC iteration.

E. Inverse Dynamics Controller and Simulation

The last stage of the controller, inverse dynamics control, is achieved using the robot dynamics

$$\tau = H(\dot{q}, q) * \ddot{q}_{des} + C(\dot{q}, q) \quad (31)$$

where $H \in \mathbb{R}^{8 \times 8}$ is the joint space inertia matrix, and $C \in \mathbb{R}^8$ is the Coriolis matrix. H and C matrices are found using the HandC function included in the package Spatial V2.

Simulation is done using a discretized model and the ode45 [13] function supplied with MATLAB. Specifically, we use a zero-order hold on the inputs, then simulate the system forward until the end of the time step. The state is then updated to the final state of the simulation, and the simulation repeated with more inputs generated from the centroidal MPC. A pseudocode of the main control and simulation loop is provided in Algorithm 1.

III. RESULTS

We tested the robot with two tasks: a 90° turn and return to the original orientation; and a 180° turn, both with a set task time of two seconds. Z axis rotation in the desired trajectory optimization state, X_{des} , is adjusted to match the desired trajectory, and all other states are set to zero. State error weight $w_{1,TO}$ is adjusted to only weigh error in Z axis rotation.

We can observe that although the trajectory optimization resulted in trajectories that appeared feasible, the dynamics used in the optimization problem was prone to integration error. The MPC controller showed a strong reliance on the tail for reorientation, which limited the reorientation range of

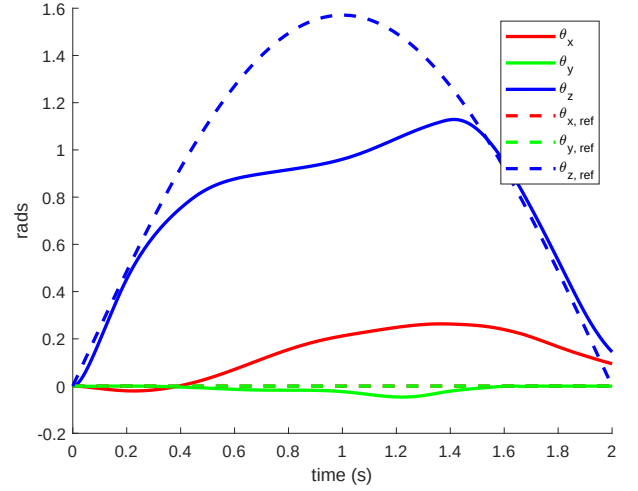


Fig. 3. X, Y, Z Euler angles for base orientation during the 90° turn movement. Dotted lines represent the reference trajectory from the offline optimization, and solid lines represent the simulated trajectory using the CM-MPC controller.

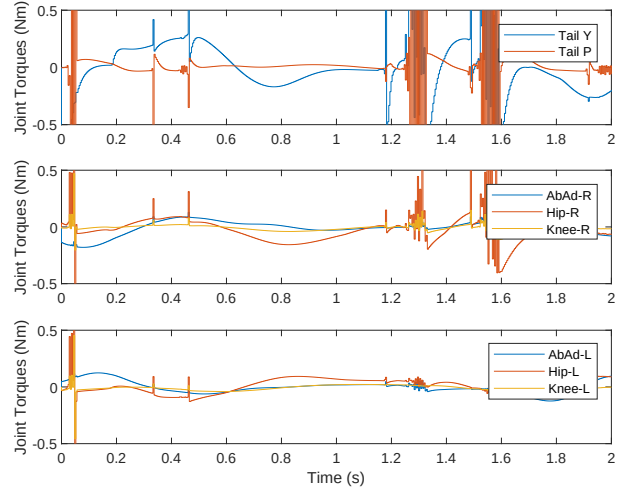


Fig. 4. Joint torques during the 90° turn movement. Top graph is the tail torques, middle graph is the right leg and bottom graph is the left leg. Oscillations are visible around 0s, 1.3s, and 1.6s, which causes clipping of motor applied torque.

motion of the base. The robot is only able to rotate its base orientation by 1.1 radians due to its joint limits in the 90° turn and return motion task, as shown in figure 3. In figure 5 it fails to turn 180° in the second motion task due to a similar reason, achieving a maximum Y axis deviation of 1.25 radians.

The controller also showed high frequency oscillations when deviating from the reference trajectory, visible in figures 4 and 6. Torques for the tail is consistently higher than that of the legs, showing that the

IV. CONCLUSION AND DISCUSSION

The centroidal momentum MPC, when applied to tailed bipedal inertial reorientation, shows promise. The robot is able to reason about the high-inertia tail and utilize it to change its base orientation without outside forces. However, the amount

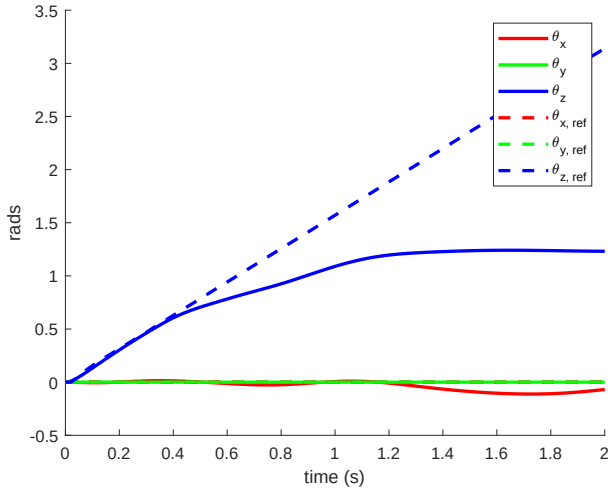


Fig. 5. X, Y, Z Euler angles for base orientation during the 90° turn movement. Reorientation reaches its maximum of 1.25 radians at around 1.2s.

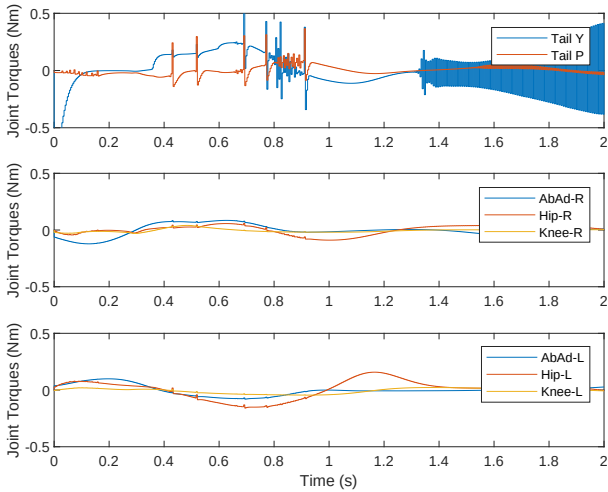


Fig. 6. Joint torques during the 180° turn movement. Graph order is similar to that in figure 4. The MPC causes severe oscillations in applied torque, especially towards the end of the motion where deviation from the reference trajectory becomes greater.

of reorientation is constrained by joint limits, and the MPC controller is not able to deduce the longer, more complex motions necessary to track large reorientation angles. Even though centroidal momentum dynamics is already a reduced order system, the CM-MPC is still limited by its computational complexity. Further streamlining of the calculation may permit the MPC to use a longer time horizon and achieve better performance.

Linearization around the reference trajectory causes the MPC controller to fail once there is a significant deviation from the reference, as evidenced by oscillations increasing in magnitude as deviation increases. Moreover, the Euler integration method used in trajectory optimization produces drift and integration error, causing further difficulties in tracking. Improvements to the whole body dynamics used in trajectory optimization may improve overall performance.

REFERENCES

- [1] A. Jusufi, D. I. Goldman, S. Revzen, and R. J. Full, “Active tails enhance arboreal acrobatics in geckos,” *Proceedings of the National Academy of Sciences*, vol. 105, no. 11, pp. 4215–4219, 2008.
- [2] T. Libby, A. M. Johnson, E. Chang-Siu, R. J. Full, and D. E. Koditschek, “Comparative design, scaling, and control of appendages for inertial reorientation,” *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1380–1398, 2016.
- [3] X. Fu, B. Zhang, C. J. Weber, K. L. Cooper, R. Vasudevan, and T. Y. Moore, “Jointed tails enhance control of three-dimensional body rotation,” *Journal of the Royal Society Interface*, vol. 22, no. 223, p. 20240355, 2025.
- [4] J. An, X. Chu, M. J. Schwaner, and K. S. Au, “Enhanced aerial reorientation performance using a 3-dof morphable inertial tail inspired by kangaroo rats,” *IEEE Robotics and Automation Letters*, 2024.
- [5] X. Chu, S. Wang, R. Ng, C. Y. Fan, J. An, and K. S. Au, “Combining tail and reaction wheel for underactuated spatial reorientation in robot falling with quadratic programming,” *IEEE Robotics and Automation Letters*, vol. 8, no. 11, pp. 7783–7790, 2023.
- [6] J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim, “Dynamic locomotion in the mit cheetah 3 through convex model-predictive control,” in *2018 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2018, pp. 1–9.
- [7] H. Dai, A. Valenzuela, and R. Tedrake, “Whole-body motion planning with centroidal dynamics and full kinematics,” in *2014 IEEE-RAS International Conference on Humanoid Robots*. IEEE, 2014, pp. 295–302.
- [8] Z. Zhou, B. Wingo, N. Boyd, S. Hutchinson, and Y. Zhao, “Momentum-aware trajectory optimization and control for agile quadrupedal locomotion,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7755–7762, 2022.
- [9] R. Featherstone, *Rigid body dynamics algorithms*. Springer, 2008.
- [10] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “CasADi – A software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [11] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.
- [12] D. E. Orin and A. Goswami, “Centroidal momentum matrix of a humanoid robot: Structure and properties,” in *2008 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2008, pp. 653–659.
- [13] L. F. Shampine and M. W. Reichelt, “The matlab ode suite,” *SIAM journal on scientific computing*, vol. 18, no. 1, pp. 1–22, 1997.